SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

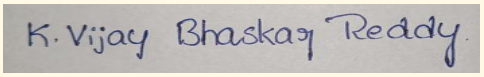
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Data Interpretation

Course Code:	**ITA0514**	**Course Title:**	**Computer Vision**
CO Assessed:	**CO4 – Articulation (combining methods for evaluation) P4**	**Assessment:**	**Assessment Tool 1 – Data Interpretation**
Weightage:	**40% of CIA**	**Total Marks:**	**100**
CO4 Statement:	*Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL5)*		
Student Name:	K.Vijay Bhaskar Reddy	**Reg. No.:**	192425155
Section / Batch:	AIML/2024	**Date:**	17/08/2026

Code of Conduct

I K.Vijay Bhaskar Reddy(192425155) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	
Clarity	10%	Presents interpretation clearly using proper	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	

		structure, visuals, and terminology				

21. Noise Impact on Detection

Given Results

Noise Level	Detection Accuracy
Low Noise	**92%**
Medium Noise	**80%**
High Noise	**60%**

Compare performance and evaluate system robustness.

Analysis

The results show that the detection system's performance decreases as the level of noise increases. Under **low noise conditions**, the system achieves a high accuracy of **92%**, indicating that it can detect objects or events reliably when the input data is relatively clean.

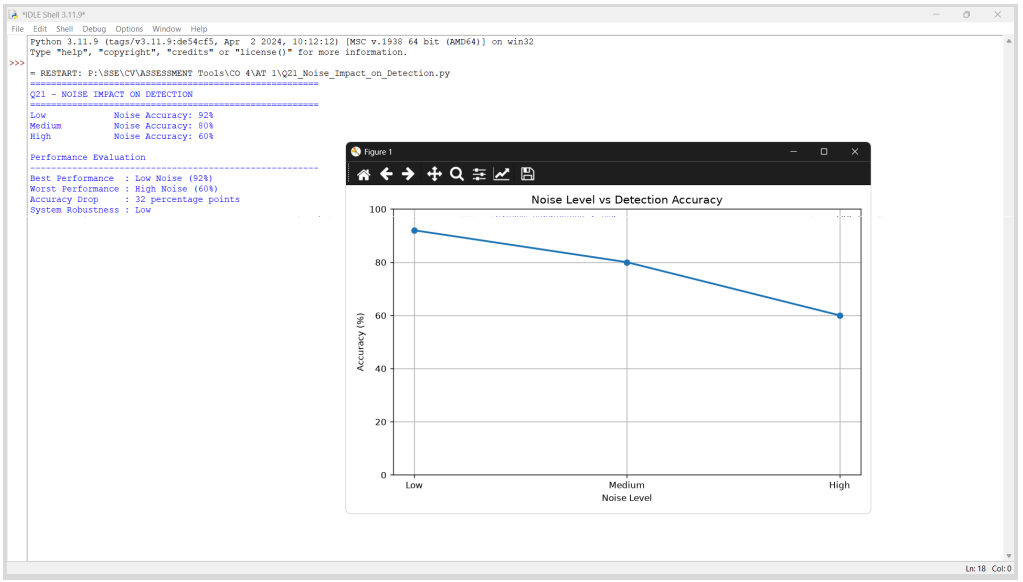
When the noise level increases to **medium**, the accuracy decreases to **80%**. This indicates that noise starts interfering with the important features required for accurate detection.

Under **high noise conditions**, the accuracy drops significantly to **60%**. The **32-percentage-point decrease** from low to high noise demonstrates that the system is sensitive to environmental noise.

Code:

```
import matplotlib.pyplot as plt

noise_levels = ["Low", "Medium", "High"]

accuracy = [92, 80, 60]

print("=" * 55)

print("Q21 - NOISE IMPACT ON DETECTION")

print("=" * 55)

for level, value in zip(noise_levels, accuracy):

    print(f"{level:<15} Noise Accuracy: {value}%")

best_index = accuracy.index(max(accuracy))

worst_index = accuracy.index(min(accuracy))

accuracy_drop = max(accuracy) - min(accuracy)

print("\nPerformance Evaluation")

print("-" * 55)

print(f"Best Performance  : {noise_levels[best_index]} Noise ({accuracy[best_index]}%)")

print(f"Worst Performance : {noise_levels[worst_index]} Noise ({accuracy[worst_index]}%)")
```

```
print(f"Accuracy Drop     : {accuracy_drop} percentage points")
if accuracy_drop <= 15:
    robustness = "High"
elif accuracy_drop <= 30:
    robustness = "Moderate"
else:
    robustness = "Low"
print(f"System Robustness : {robustness}")
plt.figure(figsize=(8, 5))
plt.plot(noise_levels, accuracy, marker="o", linewidth=2)
plt.title("Noise Level vs Detection Accuracy")
plt.xlabel("Noise Level")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
plt.grid(True)
plt.tight_layout()
plt.show()
```



Comparison

Condition	Accuracy	Performance
Low Noise	92%	Excellent
Medium Noise	80%	Good

High Noise	60%	Poor

The system performs best under low-noise conditions and worst under high-noise conditions.

Evaluation

The system has **good performance in clean and moderately noisy environments**, but its robustness decreases considerably in highly noisy environments. This can be improved using:

- Noise filtering techniques
- Image/audio denoising
- Data augmentation
- Feature extraction
- Noise-resistant machine learning models

Real-World Application

Noise robustness is important in applications such as:

- **Surveillance systems**
- **Speech recognition**
- **Autonomous vehicles**
- **Industrial monitoring**
- **Medical imaging**

For example, a surveillance camera operating near a busy road may experience noise and reduced detection accuracy. Therefore, noise-resistant preprocessing is important.

Conclusion

The system achieves **92% accuracy at low noise** but falls to **60% at high noise**. Therefore, the system is **not fully robust to high noise conditions**. Additional noise-reduction and preprocessing techniques should be implemented to maintain reliable detection in real-world environments.

22. Depth Estimation Error Analysis

Given Results

Method	Depth Estimation Error
Method A	**0.5 m**
Method B	**1.2 m**
Method C	**0.8 m**

Compare and evaluate accuracy for real-world applications.

Analysis

The depth estimation error represents the difference between the estimated depth and the actual depth. **Lower error indicates better accuracy**.

Method A produces an error of only **0.5 m**, which is the lowest among all three methods. Therefore, Method A provides the most accurate depth estimation.

Method C has an error of **0.8 m**, which is higher than Method A but lower than Method B.

Method B has the highest error of **1.2 m**, indicating the lowest depth estimation accuracy among the three methods.

Comparison

Method	Error	Accuracy Evaluation
A	**0.5 m**	Excellent
C	**0.8 m**	Good
B	**1.2 m**	Poor

The ranking is:

Method A > Method C > Method B

where Method A provides the best accuracy.

Code:

```
import matplotlib.pyplot as plt

methods = ["Method A", "Method B", "Method C"]

errors = [0.5, 1.2, 0.8]

print("=" * 55)

print("Q22 - DEPTH ESTIMATION ERROR ANALYSIS")

print("=" * 55)
```

```
for method, error in zip(methods, errors):
    print(f"{method:<15} Error: {error:.1f} m")
best_index = errors.index(min(errors))
worst_index = errors.index(max(errors))
print("\nPerformance Evaluation")
print("-" * 55)
print(f"Most Accurate Method : {methods[best_index]}")
print(f"Lowest Error         : {errors[best_index]:.1f} m")
print(f"Highest Error        : {errors[worst_index]:.1f} m")
print(f"Error Difference     : {errors[worst_index] - errors[best_index]:.1f} m")
print("\nConclusion")
print(f"{methods[best_index]} is the best method because it has the lowest")
print("depth estimation error and therefore provides the highest accuracy.")
plt.figure(figsize=(8, 5))
bars = plt.bar(methods, errors)
plt.title("Depth Estimation Error Comparison")
plt.xlabel("Method")
plt.ylabel("Error (m)")
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```

Evaluation

Method A is the most reliable method because it has the **minimum estimation error**. A difference of 0.5 m can be acceptable for many applications, depending on the required precision.

Method B may not be suitable for applications where accurate distance measurements are essential because its error is **1.2 m**.

Real-World Applications

Depth estimation is widely used in:

- **Autonomous vehicles**
- **Robotics**
- **3D reconstruction**
- **Augmented Reality**
- **Object detection**
- **Navigation systems**

For autonomous vehicles, inaccurate depth estimation could result in incorrect distance measurement from obstacles. Therefore, a method with lower error is preferred.

Conclusion

Method A is the best-performing method because it has the lowest depth estimation error of **0.5 m**. Method C provides moderate performance, while Method B has the highest error. Therefore, Method A is the most suitable for applications requiring accurate real-world depth estimation.

23. Multi-Model Comparison

Given Results

Model	Accuracy	Speed	Robustness
A	88%	Fast	Low
B	92%	Moderate	High
C	95%	Slow	High

Compare and evaluate the best overall model.

Analysis

The three models provide different trade-offs between **accuracy, processing speed, and robustness**.

Model A has the fastest processing speed, which makes it attractive for real-time applications. However, its accuracy is only **88%**, and its robustness is low.

Model B provides **92% accuracy**, moderate speed, and high robustness. Therefore, it provides a balanced performance across all three criteria.

Model C achieves the highest accuracy of **95%** and high robustness. However, it has a slow processing speed, which may be a disadvantage in real-time applications.

Comparison

Model	Accuracy	Speed	Robustness	Overall Evaluation
A	88%	Fast	Low	Suitable for speed
B	92%	Moderate	High	**Best balance**
C	95%	Slow	High	Best accuracy

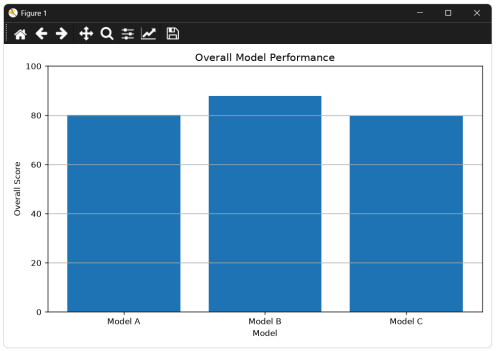
Code:

```
import matplotlib.pyplot as plt

models = ["Model A", "Model B", "Model C"]

accuracy = [88, 92, 95]

speed = ["Fast", "Moderate", "Slow"]

robustness = ["Low", "High", "High"]

speed_score = {

   "Fast": 100,
```

```
    "Moderate": 70,
    "Slow": 40
}
robustness_score = {
    "High": 100,
    "Low": 50
}
overall_scores = []
print("=" * 65)
print("Q23 - MULTI-MODEL COMPARISON")
print("=" * 65)
print(f"{'Model':<12}{'Accuracy':<12}{'Speed':<12}{'Robustness':<15}{'Score'}")
print("-" * 65)
for model, acc, spd, rob in zip(models, accuracy, speed, robustness):
    score = (0.40 * acc) + (0.30 * speed_score[spd]) + (0.30 * robustness_score[rob])
    overall_scores.append(score)
    print(f"{model:<12}{acc:<12}%{spd:<12}{rob:<15}{score:.2f}")
best_index = overall_scores.index(max(overall_scores))
print("\nPerformance Evaluation")
print("-" * 65)
print(f"Best Overall Model : {models[best_index]}")
print(f"Overall Score      : {overall_scores[best_index]:.2f}")
print(f"Highest Accuracy   : {models[accuracy.index(max(accuracy))]} ({max(accuracy)}%)")
print("\nConclusion")
print(f"{models[best_index]} provides the best balance of accuracy, speed,")
print("and robustness based on the selected evaluation weights.")
plt.figure(figsize=(8, 5))
```

```
plt.bar(models, overall_scores)

plt.title("Overall Model Performance")

plt.xlabel("Model")

plt.ylabel("Overall Score")

plt.ylim(0, 100)

plt.grid(axis="y")

plt.tight_layout()

plt.show()
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: P:\SSE\CV\ASSESSMENT Tools\CO 4\AT 1\Q23_Multi_Model_Comparison.py
============================================================
Q23 - MULTI-MODEL COMPARISON
============================================================
Model        Accuracy    Speed       Robustness     Score
------------------------------------------------------------
Model A      88          %Fast       Low            80.20
Model B      92          %Moderate   High           87.80
Model C      95          %Slow       High           80.00

Performance Evaluation
------------------------------------------------------------
Best Overall Model : Model B
Overall Score      : 87.80
Highest Accuracy   : Model C (95%)

Conclusion
Model B provides the best balance of accuracy, speed,
and robustness based on the selected evaluation weights.
```



Evaluation

If the main requirement is **maximum accuracy**, Model C is the best choice.

If the main requirement is **fast processing**, Model A is preferable.

However, if the system requires a balance between accuracy, speed, and robustness, **Model B is the most appropriate**.

Application

Different models can be selected according to application requirements:

- **Model A:** Real-time lightweight applications
- **Model B:** General-purpose intelligent systems
- **Model C:** High-accuracy offline or computationally powerful systems

For example, a real-time surveillance system may prefer Model B because it provides high accuracy without sacrificing too much processing speed.

Conclusion

Model B is the best overall model because it provides a good balance between **92% accuracy, moderate processing speed, and high robustness**. Although Model C has higher accuracy, its slow speed makes it less suitable for real-time applications.

24. Motion Detection Under Crowd Density

Given Results

Method	Sparse Crowd	Dense Crowd
Optical Flow	High	Low
Motion Models	Moderate	High

Compare and evaluate best approach.

Analysis

Motion detection performance changes significantly depending on crowd density.

In a **sparse crowd**, individuals are separated from each other, making their movement easier to track. Therefore, **Optical Flow performs well** in sparse environments.

However, when the crowd becomes dense, people overlap and their movements become more complex. This reduces the effectiveness of Optical Flow, resulting in low performance.

Motion Models show **moderate performance in sparse crowds** but perform highly in dense crowds. This indicates that Motion Models are better at handling complex movement patterns and interactions between people.

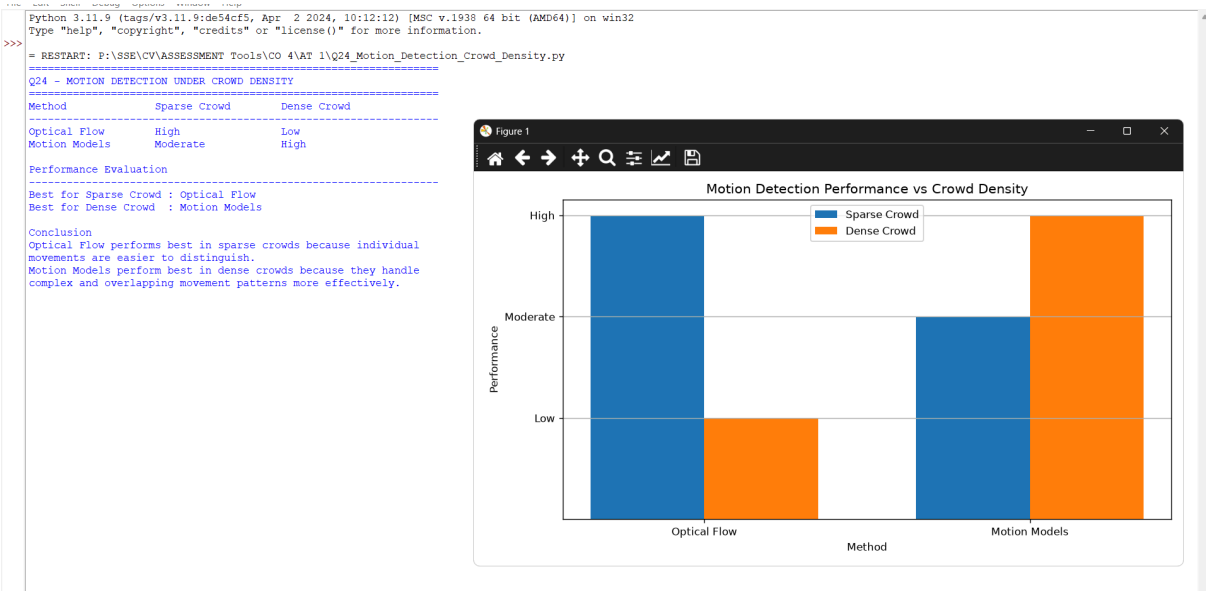
Comparison

Method	Sparse Crowd	Dense Crowd	Best Environment
Optical Flow	High	Low	Sparse crowds
Motion Models	Moderate	High	Dense crowds

Code:

```
import matplotlib.pyplot as plt
methods = ["Optical Flow", "Motion Models"]
sparse_scores = [3, 2]
dense_scores = [1, 3]
rating_names = {
    3: "High",
    2: "Moderate",
    1: "Low"
}
print("=" * 65)
print("Q24 - MOTION DETECTION UNDER CROWD DENSITY")
print("=" * 65)
print(f"{'Method':<20}{'Sparse Crowd':<20}{'Dense Crowd'}")
print("-" * 65)
for i, method in enumerate(methods):
    print(
        f"{method:<20}"
```

```
        f"{rating_names[sparse_scores[i]]:<20}"
        f"{rating_names[dense_scores[i]]}"
    )
best_sparse_index = sparse_scores.index(max(sparse_scores))
best_dense_index = dense_scores.index(max(dense_scores))
print("\nPerformance Evaluation")
print("-" * 65)
print(f"Best for Sparse Crowd : {methods[best_sparse_index]}")
print(f"Best for Dense Crowd  : {methods[best_dense_index]}")
print("\nConclusion")
print("Optical Flow performs best in sparse crowds because individual")
print("movements are easier to distinguish.")
print("Motion Models perform best in dense crowds because they handle")
print("complex and overlapping movement patterns more effectively.")
x = range(len(methods))
width = 0.35
plt.figure(figsize=(9, 5))
plt.bar([i - width / 2 for i in x], sparse_scores, width, label="Sparse Crowd")
plt.bar([i + width / 2 for i in x], dense_scores, width, label="Dense Crowd")
plt.xticks(list(x), methods)
plt.yticks([1, 2, 3], ["Low", "Moderate", "High"])
plt.title("Motion Detection Performance vs Crowd Density")
plt.xlabel("Method")
plt.ylabel("Performance")
plt.legend()
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```



Evaluation

Optical Flow is effective when there are fewer objects and less movement overlap. It can provide detailed information about the direction and magnitude of motion.

However, in crowded environments, the movement of individuals becomes difficult to distinguish. In such cases, **Motion Models perform better** because they can represent more complex motion patterns.

Real-World Applications

Motion detection under crowd density is important in:

- **Crowd surveillance**
- **Public safety systems**
- **Stadium monitoring**
- **Railway station monitoring**
- **Airport security**
- **Event management**

For example, during a large public event, the crowd may become highly dense. A motion detection system must therefore be capable of handling overlapping and complex movements.

Conclusion

Motion Models are the better overall approach, especially for **dense crowd environments**. Optical Flow is more suitable for sparse crowds because it provides high performance when individual movements are clearly visible. Therefore, the choice of method should depend on the crowd density and application requirements.

25. Model Efficiency vs Accuracy

Given Results

Model	Accuracy	Computational Requirement
A	90%	High
B	85%	Low
C	88%	Moderate

Compare and evaluate for edge devices.

Analysis

The results demonstrate a trade-off between **model accuracy and computational efficiency**.

Model A provides the highest accuracy of **90%**, but it requires high computational resources. This means that it may require a powerful processor, more memory, and greater energy consumption.

Model B requires low computational resources, making it highly efficient. However, its accuracy is only **85%**, which is the lowest among the three models.

Model C achieves **88% accuracy** while requiring only moderate computational resources. Therefore, it provides a reasonable compromise between accuracy and efficiency.

Comparison

Model	Accuracy	Computation	Suitability for Edge Devices
A	90%	High	Low
B	85%	Low	Excellent
C	88%	Moderate	**Very Good**

Evaluation

Edge devices such as **IoT devices, smartphones, embedded systems, drones, and smart cameras** have limited computational resources.

Although Model A provides the highest accuracy, its high computational requirement may cause:

- Higher power consumption
- Increased processing time
- More heat generation
- Greater hardware requirements

Model B is highly efficient but sacrifices some accuracy.

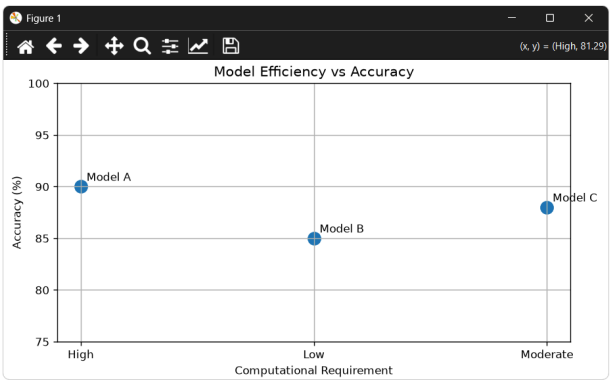
Model C provides **88% accuracy with moderate computation**, making it a good balance for edge deployment.

Code:

```
import matplotlib.pyplot as plt
models = ["Model A", "Model B", "Model C"]
accuracy = [90, 85, 88]
computation = ["High", "Low", "Moderate"]
computation_score = {
    "Low": 1,
    "Moderate": 2,
    "High": 3
}
print("=" * 65)
print("Q25 - MODEL EFFICIENCY VS ACCURACY")
print("=" * 65)
print(f"{'Model':<12}{'Accuracy':<12}{'Computation':<15}")
print("-" * 65)
for model, acc, comp in zip(models, accuracy, computation):
    print(f"{model:<12}{acc:<12}%{comp:<15}")
efficiency_score = []
for acc, comp in zip(accuracy, computation):
    computation_efficiency = {
        "Low": 100,
        "Moderate": 70,
        "High": 40
    }[comp]

    score = (0.70 * acc) + (0.30 * computation_efficiency)
    efficiency_score.append(score)
best_index = efficiency_score.index(max(efficiency_score))

print("\nEdge Device Evaluation")
print("-" * 65)
```

```
for model, score in zip(models, efficiency_score):
    print(f"{model:<12} Edge Suitability Score: {score:.2f}")

print("\nConclusion")
print(f"{models[best_index]} is the most suitable model for edge devices")
print("because it provides a good balance between accuracy and computation.")

plt.figure(figsize=(8, 5))
plt.scatter(computation, accuracy, s=120)

for i, model in enumerate(models):
    plt.annotate(
        model,
        (computation[i], accuracy[i]),
        xytext=(5, 5),
        textcoords="offset points"
    )

plt.title("Model Efficiency vs Accuracy")
plt.xlabel("Computational Requirement")
plt.ylabel("Accuracy (%)")
plt.ylim(75, 100)
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
= RESTART: P:\SSE\CV\ASSESSMENT Tools\CO 4\AT 1\Q25_Model_Efficiency_vs_Accuracy.py
==========================================================================
Q25 - MODEL EFFICIENCY VS ACCURACY
==========================================================================
Model       Accuracy    Computation
--------------------------------------------------------------------------
Model A     90          %High
Model B     85          %Low
Model C     88          %Moderate

Edge Device Evaluation
--------------------------------------------------------------------------
Model A      Edge Suitability Score: 75.00
Model B      Edge Suitability Score: 89.50
Model C      Edge Suitability Score: 82.60

Conclusion
Model B is the most suitable model for edge devices
because it provides a good balance between accuracy and computation.
```



Real-World Application

Model efficiency is particularly important for:

- **Smart cameras**
- **IoT devices**
- **Mobile applications**
- **Drones**
- **Autonomous robots**
- **Wearable devices**
- **Embedded systems**

For example, a smart camera running continuously on an edge device needs a model that provides sufficient accuracy without consuming excessive processing power.

Conclusion

Model C is the most suitable overall model for edge devices because it provides a good balance between **88% accuracy and moderate computational requirements**.

Model B should be selected when **minimum computation and power consumption** are the highest priorities. Model A should be selected when **maximum accuracy** is more important than computational efficiency.